

Lab 4: Mini Event-Driven App – Email Notification System

Goal

Build a simple event-driven system that demonstrates how to combine delegates, lambda expressions, and events. Simulate a user registration process that triggers notifications.

Step-by-Step Instructions

Step 1: Create the Project

1. Open Visual Studio or your preferred C# development environment.
 2. Create a new Console App project named `EventDrivenNotifierLab`.
-

Step 2: Define the User Class

```
public class User
{
    public string Name { get; set; }
    public string Email { get; set; }
}
```

Step 3: Define the UserRegistered EventArgs

```
public class UserRegisteredEventArgs : EventArgs
{
    public User NewUser { get; set; }
}
```

Step 4: Create the Registration Service

```

public class RegistrationService
{
    public event EventHandler<UserRegisteredEventArgs> UserRegistered;

    public void RegisterUser(string name, string email)
    {
        var user = new User { Name = name, Email = email };
        Console.WriteLine($"Registering user: {name}");
        UserRegistered?.Invoke(this, new UserRegisteredEventArgs { NewUser = user });
    }
}

```

✅ Step 5: Implement Notification Handlers

```

public class Logger
{
    public void Subscribe(RegistrationService regService)
    {
        regService.UserRegistered += (s, e) =>
        {
            Console.WriteLine($"[LOG] User registered: {e.NewUser.Name}
({e.NewUser.Email})");
        };
    }
}

public class WelcomeEmailSender
{
    public void Subscribe(RegistrationService regService)
    {
        regService.UserRegistered += (s, e) =>
        {
            Console.WriteLine($"[EMAIL] Sent welcome email to {e.NewUser.Email}");
        };
    }
}

```

✅ Step 6: Tie It All Together

In `Main()`:

```
var regService = new RegistrationService();
var logger = new Logger();
var emailSender = new WelcomeEmailSender();

logger.Subscribe(regService);
emailSender.Subscribe(regService);

regService.RegisterUser("Alice", "alice@example.com");
regService.RegisterUser("Bob", "bob@example.com");
```

🌟 Stretch Goal: Add Custom Behavior with a Delegate

Define a delegate and inject extra registration logic:

```
delegate void ExtraAction(User user);

public class RegistrationService
{
    public event EventHandler<UserRegisteredEventArgs> UserRegistered;
    public ExtraAction AdditionalAction;

    public void RegisterUser(string name, string email)
    {
        var user = new User { Name = name, Email = email };
        Console.WriteLine($"Registering user: {name}");
        AdditionalAction?.Invoke(user);
        UserRegistered?.Invoke(this, new UserRegisteredEventArgs { NewUser = user });
    }
}
```

Assign a lambda to `AdditionalAction`:

```
regService.AdditionalAction = u => Console.WriteLine($"[CUSTOM] Additional processing for {u.Name}");
```

✅ What You Learned

- How to use events to decouple logic
- How to wire up multiple subscribers
- How to combine lambdas and delegates in real-world apps